

DATA/CS 172: Python for Data Analysis

Tuples

Prof. Travis Mandel

9/20/2024



UNIVERSITY
of HAWAII®

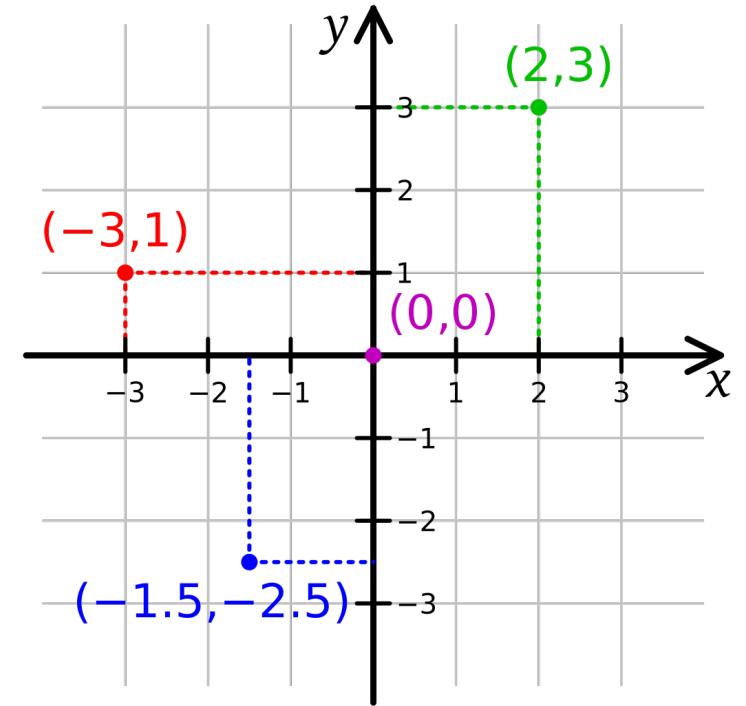
HILO

Reminders

- Program 1 out Monday
 - Sorry for delay
- Keep on top of labs

Tuples

- Like lists but they can't be changed
- List: $[4,7,8]$
- Tuple: $(4,7,8)$
 - Note: Parentheses, not square brackets
 - Tuples of two elements sometimes called pairs
 - Tuples of three elements sometimes called triples
 - Can have any number of elements
- Tuple of zero elements: $()$
- Tuple of one element: $(4,)$
 - Huh?
 - Couldn't be (4) as $(4)+6$ should be 10!
 - Very confusing, especially since they didn't do this with zero elements!



Can do most list things with tuples

- **Can** get the length with `len(myTuple)`
- **Can** slice with `myTuple[x:y]`
- **Can** get a certain index with the indexing operator
 - `secondItem = myTuple[1]`
- But...
 - The data is locked in
 - **Cannot** assign! `myTuple[1] = 6`
 - **Cannot** append! `myTuple.append(7)`



Mixed Tuples

- Tuples often used to store a small number of closely related things, often of **different types**.
- Example:
 - (“John”, 93, “Freshman”)
 - (“Emma”, 80, “Sophomore”)
- First represents name, then grade, then standing



Unpacking tuples

- `myTuple = ("John", 93, "Freshman")`
- How to unpack this? The **bad** way:
 - `name = myTuple[0]`
 - `grade = myTuple[1]`
 - `standing = myTuple[2]`
 - Ugly! Confusing! Hard to understand!
- The **much** better way:
 - `(name, grade, standing) = myTuple`
 - Yes, we can have multiple things on the left hand side!
 - Works perfectly!
 - Even if you just need the name, best to do it this way so it's clear what each thing in the tuple represents



Tuples or lists?



- Tuples

- Nothing can be added, changed, or removed (**immutable**) ← Important term
- Most commonly used when there is a small number of items, each with different meanings
- Indexing (e.g. `myTuple[2]`) is usually bad

- Lists

- Can add, change, remove (**mutable**) ← Important term
- Most commonly used when there is a large number of similar items in a sequence
- Indexing (e.g. `myList[2]`) is usually good

Converting between lists and tuples

- Use our standard type conversion stuff!
- `list((1,2,3))`
 - `[1,2,3]`
- `tuple([1,2,3])`
 - `(1,2,3)`
- Useful if you need to change a tuple for some reason
 - Convert to list, then convert back



Combining lists and tuples

- Can have lists of tuples
- Example
 - [(“Allan”, 3.8, “CS”), (“Paige”, 3.9, “ASTR”)]
- Fine to loop through as normal:
 - for studentTuple in students:
 - (name, gpa, major) = studentTuple
 - #do something
- A little bit shorter and nicer:
 - for (name, gpa, major) in students:
 - #do something

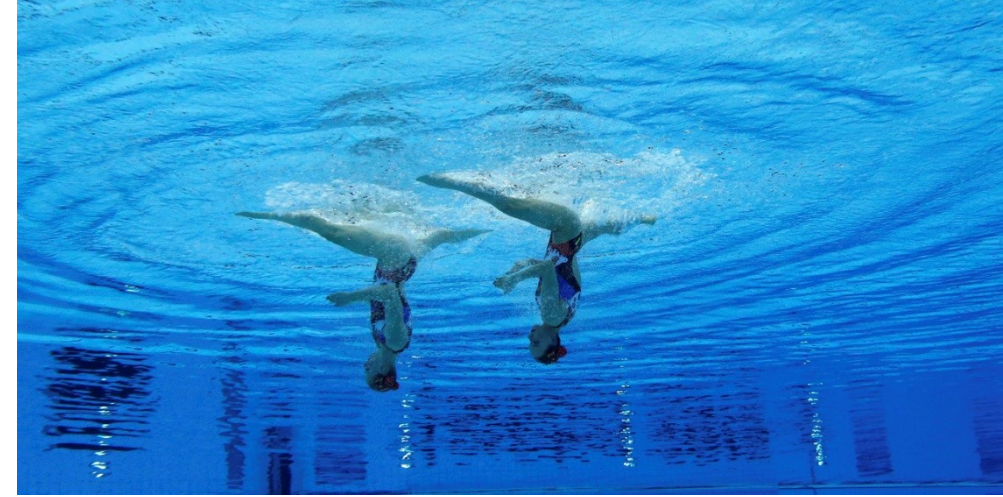


What if I have two lists, that I want to go through at the same time?

- Old way:
 - `for i in range(len(names)):`
 - `print(names[i])`
 - `print(grades[i])`
- Perfectly fine, but a little clunky
- Can tuples help us do this better?

Simultaneous for loop

- for (name, grade) in zip(names, grades):
 - print("Name is ", name)
 - print("Grade is ", grade)
- Goes through the first in both, then the second in both, etc.
 - No need to worry about indices etc.
- Zip takes two lists and zips them
 - zip([1,2,3,4], [5,6,7,8]) → [(1,5), (2,6), (3,7), (4,8)]
 - Don't overuse – most useful here



Next lab out

- Brief practice with tuples